

HelixQA

HelixQA

블러핑 방지 QA 오케스트레이션 — 모든 **PASS**에 실제 사용자가 기능을 사용할 수 있다는 증거가 포착된 자율 크로스 플랫폼 세션

요약

HelixQA은 크로스 플랫폼 테스트(Android, Android TV, Web, Desktop)를 위한 블러핑 방지 QA 오케스트레이션 프레임워크로, YAML 테스트 뱅크, 실시간 크래시 감지, 단계별 증거 포착, LLM 기반 컴퓨터 비전 자율 QA 세션을 결합하여 기능이 실제로 엔드투엔드로 작동함을 입증합니다. 이는 Constitution의 필수 QA 테스트 유형(\$11.4.169)입니다.

간략 설명

블러핑 방지 QA 오케스트레이터(Go)로, 작성된 테스트 뱅크와 완전 자율적인 LLM 및 비전 기반 QA 세션을 플랫폼 간에 실행합니다. 크래시를 감지하고, 각 단계의 증거(스크린샷, 로그캣, 영상, 스택 트레이스)를 검증하며, 증거가 풍부한 티켓을 자동 생성하여 AI 수정 파이프라인으로 전달합니다.

상세 설명

HelixQA은 Go 프레임워크로, Constitution의 §11.4 운영 규칙이라는 단 하나의 확고한 설계 원칙을 따릅니다. 즉, 출시를 결정하는 기준은 “테스트 통과”가 아니라 “사용자가 기능을 사용할 수 있음”이며, 모든 **PASS** 결과에는 실행 중에 포착된 확실한 증거가 반드시 포함되어야 합니다. 증거가 없으면 **PASS**도 없고, 예외도 없습니다. 이 프레임워크는 서로 보완적인 두 가지 모드를 실행하여 스크립트 기반과 미지의 상황 모두를 커버합니다.

첫째, 작성된 테스트 뱅크 — 플랫폼 타겟팅, 우선순위, 순서화된 단계(이름/액션/예상 결과), 태그, 문서 참조를 포함한 TC-xxx 케이스로 구성된 YAML 테스트 스위트입니다. 각 단계별 검증, 실시간 크래시/ANR 감지(Android의 경우 ADB, 웹/데스크톱의 경우 프로세스 모니터링), 중앙 집중식 증거 수집, 그리고 하류 AI 수정 파이프라인에 맞춰 자동 생성된 마크다운 티켓으로 실행됩니다.

둘째, 완전 자율 **QA** 세션 — LLM 기반 에이전트와 컴퓨터 비전에 앱을 맡기고, 네 가지 엄격한 단계에 따라 무인으로 실행됩니다. 설정 단계(LLM 선택, 프로젝트 문서를 기반으로 기능 맵 구축, CLI 에이전트 생성, 비전 엔진 초기화), 문서 기반 검증(모든 문서화된 기능 확인), 호기심 기반 탐색(의도적으로 엇지 케이스와 미문서화된 동작을 테스트), 그리고 마지막으로 모든 발견 사항을 비디오 타임스탬프 증거와 함께 마크다운/HTML/JSON 형식으로 보고 및 정리합니다.

핵심은 스스로의 숙제를 채점하지 않는다는 점입니다. 네 가지 외부 Go 하위 모듈(LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor)을 통합하고, 공유 challenges 및 containers 인프라를 재사용하여 앱을 탐색하는 구성 요소와 그 결과가 유효한지 판단하는 구성 요소가 분리되어 있습니다. 자체 테스트 스위트도 make anti-bluff(정적 스캔 + 행동 앵커 매니페스트 + 뮤테이션 래칭)와 내장된 §1.1 뮤테이션을 포함한 8단계 오케스트레이터 챌린지를 통해 다른 제품과 동일한 기준을 적용받습니다. 15개 행으로 구성된 테스트 유형 커버리지 매트릭스는 모든 광고된 기능을 구체적인 실행 가능한 자산과 특정 증거 형태로 연결하여, 프레임워크가 스스로에 대해 주장하는 내용도 테스트하는 제품에 내리는 판정만큼 증거에 기반하도록 보장합니다.

콘텐츠

왜 만들었는가

기존의 QA는 “단언문이 통과했다”는 것만으로 녹색불이 켜집니다. 바로 이런 방식으로 Constitution가 허위 보고라고 부르는 실패 유형이 그대로 통과되는 것이죠. 겉으로는 작동하는 것처럼 보이지만 실제 사용자에게는 문제가 있는 기능 말입니다. HelixQA는 이런 상황을 QA 차원에서 근본적으로 차단하기 위해 만들어졌습니다. 실제 실행 환경에서 캡처된 물리적 증거(스크린샷, 로그캣, 동영상, 스택 트레이스, 리포트)가 없으면 PASS를 인정하지 않으며, 증거 없는 녹색 요약 라인은 누락된 기능과 동급의 치명적 결함으로 간주합니다. 또한 HelixQA는 여러 플랫폼에 걸친 종합적인 수동 QA가 확장되지 않는 노동력 문제를 해결합니다. 세션을 완전히 자동화함으로써 말이죠.

왜 혁신적인가

이 도구는 거의 함께 존재하지 않았던 두 가지를 결합했습니다. 바로 엄격한 증거 기반 QA 게이트와 자율적 자기 주도 탐색입니다. LLM와 비전 에이전트가 실제 앱을 열고, 문서화된 모든 기능을 검증하며, 아무도 테스트를 작성하지 않은 미문서화 버그를 찾아내고, 그 과정에서 법정 수준의 증거 자료를 생성합니다. 이제 “테스트를 진행했습니다”라는 말은 “여기 동영상, 여기 로그캣, 여기 티켓이 있습니다”로 대체됩니다. 게다가 Constitution가 명명한 QA 하위 모듈이기 때문에, 이를 도입하면 한 팀의 QA 신뢰성만 높아지는 것이 아닙니다. 한 번의 조치로 제품군 전체의 기준을 끌어올립니다.

혁신적인 점

- 허위 보고 방지 증거 계약 — 모든 체크의 PASS는 실행 시 캡처된 증거와 연동되며, 녹색 CI 라인은 필요 조건이지만 절대 충분 조건이 아닙니다. 증거 없는 녹색 요약은 치명적 결함으로 처리됩니다.
- 자율적 문서 기반 + 호기심 기반 탐색 — 문서화된 모든 기능을 검증한 후, 실제 사용자가 마주치는 엣지 케이스(빈 입력, 빠른 상호작용, 미문서화 경로 등)를 탐색하며, 수동으로 작성된 테스트 스위트에서는 예상하지 못한 부분을 찾아냅니다.
- 비전 오라클 — GoCV 기계 비전과 LLM 비전 API가 실행 중인 UI를 실제로 보면서, 토큰 및 속성 수준 단언문으로는 놓치는 시각적 결함을 잡아냅니다.
- 구조 기반 테스트 बैं크 — बैं크 문자열은 구조를 설명하며, 런타임에 LLM가 생성하는 질문 프롬프트를 유도합니다(CONST-046). 따라서 UI 텍스트가 번역되어도 하나의 बैं크로 여러 로케일을 지원할 수 있습니다.
- **AI** 수정 파이프라인용 티켓 — 자동 생성된 마크다운 이슈에는 모든 증거 자료가 첨부되어 있어, 인간 트리아저가 아닌 하류 수정 에이전트에 곧바로 전달할 수 있습니다.

모든 제품에서 활용되는 방식(제공하는 기능)

필수 품질 기둥으로서(Constitution §11.4.169에서 helix_qa 하위 모듈을 필수 테스트 유형 중 하나로 명시), HelixQA는 제품군 내 모든 제품에 동일한 기능을 제공합니다.

- 자율 **QA** 세션 — `helixqa autonomous --project ... --platforms android,desktop,web` 명령 하나로 LLM와 비전 에이전트가 실제 앱을 무인으로 구동하며 커버리지 목표를 향해 자동으로 테스트를 진행합니다. 보고서, 티켓, 동영상을 생성하는 과정에서 인간 개입은 필요 없습니다.
- 테스트 बैं크/스위트 — YAML बैं크(라운드-219 기준 최소 30개 이상)는 플랫폼별로 타겟팅되고 우선순위가 부여되며, 검증하는 문서와 줄 단위로 추적 가능합니다.
- 캡처된 증거 자료 — 스크린샷, 로그캣, 동영상, 스택 트레이스, 전체 타임라인이 중앙화되어 모든 보고서와 연결됩니다. 따라서 모든 판정은 사후 재생 및 감사가 가능합니다.
- 독립적 판정 (§11.4.141 독립 원칙) — LLM 기반 issuedetector와 비전 오라클은 앱의 동작을 탐색 에이전트와 독립적으로 판단하며, 시스템이 자신의 작업을 스스로 올바르게 평가하는 고전적 실패를 구조적으로 차단합니다.
- 게이트 + 변이 래치 — `make qa-all / make anti-bluff` 및 `challenges/scripts/helixqa_orchestrator_challenge.sh`(8단계, 내장 §1.1 변이)는 HelixQA 자체의 신뢰성을 지속적으로 검증합니다. 또한 마감 압박 속에서도 규율을 해제할 수 있는 `--skip-helixqa`와 같은 탈출구는 의도적으로 존재하지 않습니다.

가장 큰 기술적 도전 과제와 해결 방법

- **QA 자체에서의 오탐 방지** — 허풍을 잡아내는 도구가 오히려 허풍이 되어서는 안 된다 → 모든 단계는 캡처된 증거로 검증되며, 증거가 없는 PASS는 PASS가 아닌 결함으로 처리된다. 또한 행동 기준 매니페스트는 각 광고된 기능을 실행 가능한 테스트(CONST-035)에 연결하여, 테스트로 검증되지 않은 기능은 주장할 수 없게 한다.
- **하나의 두뇌로 이기종 플랫폼 제어** — Android, Android TV, Web, Desktop는 입력 모델을 공유하지 않는다 → 단일 navigator 패키지가 플랫폼별 ActionExecutor(ADB, Playwright, X11)와 플랫폼별 크래시 감지기(android/web/desktop)를 추상화하여, 오케스트레이션 로직은 한 번만 작성되고 플랫폼 차이는 가장자리에서만 처리된다.
- **자율 에이전트를 유용하게, 혼돈스럽게 만들지 않기** — 감독 없는 LLM가 앱 안에서 영원히 헤맬 수 있다 → LLMsVerifier가 적합한 모델을 평가하고 선택하며, LLMOrchestrator가 헤드리스 CLI 에이전트(opencode, claude-code, gemini, junie, qwen-code)를 관리한다. DocProcessor는 탐색의 목표를 제공하는 기능 맵을 구축하고, VisionEngine는 모든 결정이 모델의 상상력이 아닌 화면의 실제 픽셀에 기반하도록 한다.
- **현지화에 안전한 테스트 बैं크** — 영어 UI 텍스트를 하드코딩한 테스트 스위트는 15개 언어로 깨진다 → 테스트 बैं크는 구조만 기술하며, 사용자에게 노출되는 프롬프트 텍스트는 LLM/리소스를 통해 런타임에 로드된다(CONST-046). 따라서 동일한 테스트 बैं크가 지역과 무관하게 동일한 동작을 검증한다.
- **게이트가 허위가 아님을 증명하기** — 스스로 실패할 수 없는 안티-허풍 게이트는 궁극의 허풍이다 → §1.1 변이 쌍은 타입의 증거 캡처 또는 안티-허풍 단언을 제거하고 게이트가 FAIL하도록 요구하며, 변이 래치는 시간이 지나도 이 보장이 조용히 약화되지 않도록 방지한다.

기술 스택

- **Go 1.24+** 오케스트레이터 — 이유: QA는 제품이 실행되는 모든 곳에서 동작해야 하므로, 단일 정적 링크된 빠르고 이식 가능한 바이너리가 런타임 의존성이 높은 대안보다 낫다. 방법: cmd/helixqa CLI가 run/list/report/autonomous/version과 같은 조합 가능한 하위 명령어를 제공한다.
- **YAML 테스트 बैं크(pkg/testbank)** — 이유: 테스트 스위트는 선언적이고 가독성이 높아야 하며, Go를 건드리지 않고도 사람이 편집할 수 있어야 한다. 방법: version/name/test_cases[] 구조로, 각 테스트 케이스는 id, category, priority, platforms, 순서가 지정된 steps[], 그리고 기능 문서로의 추적성을 위한 documentation_refs[]를 포함한다.
- **크래시/ANR 감지기(pkg/detector)** — 이유: 가장 중요한 실패는 사후 검증이 아닌, 상호작용 중 실시간으로 발생하는 것이다. 방법: Android는 ADB(pidof/logcat/screenshot)를, 웹/데스크톱은 pgrep를 사용하여 테스트가 실행되는 동안 프로세스를 감시한다.
- **증거 수집(pkg/evidence, pkg/session)** — 이유: 안티-허풍 계약은 모든 PASS에 물리적 증거가 뒷받침될 때만 유효하다. 방법: 스크린샷, logcat, 비디오, 스택 트레이스를 SessionRecorder 타임라인으로 캡처하여 모든 리포트가 이를 참조하도록 한다.

- 자율 세션(**pkg/autonomous, pkg/navigator, pkg/issuedetector**) — 이유: 네 가지 플랫폼에 걸친 포괄적인 수동 QA는 확장성이 없으므로, 탐색 자체가 스스로 진행되어야 한다. 방법: 4단계 SessionCoordinator와 ActionExecutor(ADB/Playwright/X11), 그리고 시각적, UX, 접근성, 기능적 결함을 포괄하는 LLM 버그 감지기를 사용한다.
- 외부 서브모듈 — 이유: 재사용과 디커플링(CONST-051), 그리고 결정적으로 내비게이터와 판별자의 분리. 방법: LLMsVerifier(모델 평가), LLMOrchestrator(헤드리스 CLI 에이전트), VisionEngine(GoCV + LLM Vision), DocProcessor(기능 맵/커버리지) 등 각각 독립적으로 관리되는 컴포넌트다.
- 안티-허풍 게이트 + 변이 래킷 — 이유: HelixQA가 다른 모든 것에 강제하는 §1.1 규약을 스스로 준수하도록 하기 위해. 방법: make anti-bluff 스캔과 행동 기준 매니페스트, 변이 래킷을 사용하며, helixqa_orchestrator_challenge.sh가 8단계 엔드투엔드 검증기로 작동한다.
- **15행** 커버리지 매트릭스(**docs/test-coverage.md**) — 이유: CONST-050(B)는 간극 없는 완전한 테스트 유형 세트를 요구한다. 방법: 각 행은 구체적인 실행 가능한 자산과 특정 형태의 캡처된 증거에 연결되어, 커버리지가 주장이 아닌 검증된 사실로 존재한다.

내용

상태 및 정직성 안내

- 상태: 베타. 활발히 개발 중(README 상태 배너 219회차). 자체 허위 방지 기준을 준수함.
- 라이선스: **Apache-2.0**. 설치 명령: go install digital.vasic.helixqa/cmd/helixqa@latest.

우선 순위 등급: Helix-주 — Helix 제품군의 필수 품질/허위 방지 기준을 충족하는지 기능이 실제로 작동하는지를 검증하는 핵심 요소.